

Here are the SQL queries corresponding to the problem statements for Sets 1 through 10. These solutions are formulated using standard ANSI SQL/Oracle syntax, which is typical for these types of academic database problems.

Set 1

Table Creation & Schema We first create the tables by identifying their primary and foreign keys.

```
CREATE TABLE Person (  
    driver_id VARCHAR(10) PRIMARY KEY,  
    name VARCHAR(50),  
    address VARCHAR(100)  
);
```

```
CREATE TABLE Car (  
    license VARCHAR(15) PRIMARY KEY,  
    model VARCHAR(30),  
    year NUMBER(4)  
);
```

```
CREATE TABLE Accident (  
    report_no VARCHAR(10) PRIMARY KEY,  
    date_acc DATE,  
    location VARCHAR(100)  
);
```

```
CREATE TABLE Owns (  
    driver_id VARCHAR(10),  
    license VARCHAR(15),  
    PRIMARY KEY (driver_id, license),  
    FOREIGN KEY (driver_id) REFERENCES Person(driver_id),  
    FOREIGN KEY (license) REFERENCES Car(license)  
);
```

```
CREATE TABLE Participated (  
    driver_id VARCHAR(10),  
    model VARCHAR(30),  
    report_no VARCHAR(10),  
    damage_amount NUMBER(10,2),  
    PRIMARY KEY (driver_id, report_no),  
    FOREIGN KEY (driver_id) REFERENCES Person(driver_id)  
);
```

Queries

```

-- 1) Insert 5 records into each table [cite: 8]
-- Example for Person table; repeat similar INSERTs for the rest.
INSERT INTO Person VALUES ('D1', 'Alice', 'Pune');
INSERT INTO Person VALUES ('D2', 'Bob', 'Mumbai');
INSERT INTO Person VALUES ('D3', 'Charlie', 'Nashik');
INSERT INTO Person VALUES ('D4', 'David', 'Nagpur');
INSERT INTO Person VALUES ('D5', 'Eve', 'Aurangabad');

-- 2) Display the Schema of each table [cite: 9]
DESCRIBE Person;
DESCRIBE Car;
DESCRIBE Accident;
DESCRIBE Owns;
DESCRIBE Participated;

-- 3) Display all records from the person table [cite: 11]
SELECT * FROM Person;

-- 4) Set a Foreign key constraint from Participated.report_no to Accident.report_no [cite: 13]
ALTER TABLE Participated
ADD CONSTRAINT fk_accident_report
FOREIGN KEY (report_no) REFERENCES Accident(report_no);

-- 5) Add check constraints on Damage_amount that amt should not be negative [cite: 18]
ALTER TABLE Participated
ADD CONSTRAINT chk_damage_amt CHECK (damage_amount >= 0);

-- 6) Write a query to make the name column in the Person table NOT NULL [cite: 15]
ALTER TABLE Person
MODIFY name VARCHAR(50) NOT NULL;

-- 7) Rename the Car table to Vehicle [cite: 17]
ALTER TABLE Car RENAME TO Vehicle;

-- 8) Create a new table named new_person from existing person table and drop it [cite: 19]
CREATE TABLE new_person AS SELECT * FROM Person;
DROP TABLE new_person;

```

Set 2

Queries

```

-- 1) Show the cname, Acc_Type, amount information of customer who is having an saving account [cite: 26]

```

```
SELECT c.Cname, a.Acc_Type, a.Amount  
FROM Customer c
```

```
JOIN Account a ON c.C_Id = a.C_Id  
WHERE a.Acc_Type = 'Saving';
```

-- 2) Display the data using Natural, left and right join [cite: 27]

-- Natural Join

```
SELECT * FROM Customer NATURAL JOIN Account;
```

-- Left Join

```
SELECT * FROM Customer c LEFT JOIN Account a ON c.C_Id = a.C_Id;
```

-- Right Join

```
SELECT * FROM Customer c RIGHT JOIN Account a ON c.C_Id = a.C_Id;
```

-- 3) Display the information of customers living in the same city as of 'pooja' [cite: 28]

```
SELECT * FROM Customer
```

```
WHERE City = (SELECT City FROM Customer WHERE Cname = 'Pooja');
```

-- 4) Display the information of account, having less amount than average amount throughout the bank [cite: 29]

```
SELECT * FROM Account
```

```
WHERE Amount < (SELECT AVG(Amount) FROM Account);
```

-- 5) Display the C_id having maximum amount in account [cite: 30]

```
SELECT C_Id FROM Account
```

```
WHERE Amount = (SELECT MAX(Amount) FROM Account);
```

-- 6) Display the amount and acc_type of those customers whose amount is the minimum amount of that Acc_type [cite: 31]

```
SELECT Amount, Acc_Type FROM Account a1
```

```
WHERE Amount = (SELECT MIN(Amount) FROM Account a2 WHERE a1.Acc_Type =  
a2.Acc_Type);
```

-- 7) Display the amount of those accounts whose amount is higher than amount of any saving account [cite: 32]

```
SELECT Amount FROM Account
```

```
WHERE Amount > ANY (SELECT Amount FROM Account WHERE Acc_Type = 'Saving');
```

Set 3

Table Creation

```
CREATE TABLE Department (  
dept_id NUMBER PRIMARY KEY,  
dept_name VARCHAR(50),
```

```

    location VARCHAR(50)
); -- [cite: 36]

CREATE TABLE Employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR(50),
    city VARCHAR(50),
    salary NUMBER(10,2),
    department NUMBER,
    FOREIGN KEY (department) REFERENCES Department(dept_id)
); -- [cite: 35]

```

Queries

```

-- 1) Display schema of each table [cite: 37]
DESCRIBE Department;
DESCRIBE Employee;

-- 2) Insert 10 records in each table [cite: 38]
-- (Provide representative queries)
INSERT INTO Department VALUES (1, 'IT', 'Pune');
INSERT INTO Employee VALUES (101, 'John', 'Pune', 50000, 1);

-- 3) Insert a new department into the Department table [cite: 39]
INSERT INTO Department VALUES (2, 'HR', 'Mumbai');

-- 4) Insert an employee record without specifying the salary column [cite: 40]
INSERT INTO Employee (emp_id, emp_name, city, department)
VALUES (102, 'Alice', 'Nashik', 2);

-- 5) Update the salary of all employees by adding 20% increment [cite: 41]
UPDATE Employee SET salary = salary * 1.20;

-- 6) Display employee name and annual salary of employees whose names having the second letter e
[cite: 42]
SELECT emp_name, (salary * 12) AS annual_salary
FROM Employee WHERE emp_name LIKE '_e%';

-- 7) Update the city to 'Pune' for employees working in the 'IT' department [cite: 43]
UPDATE Employee
SET city = 'Pune'
WHERE department = (SELECT dept_id FROM Department WHERE dept_name = 'IT');

```

Set 4

Assuming tables are created similarly to Set 3.

Queries

-- 1) Display schema of each table [cite: 47]

```
DESCRIBE Employee;
```

```
DESCRIBE Department;
```

-- 2) Insert 10 records in each table [cite: 48]

-- Same approach as Set 3, query 2.

-- 3) Update the city to 'Pune' for employees working in the 'IT' department [cite: 49]

```
UPDATE Employee
```

```
SET city = 'Pune'
```

```
WHERE department = (SELECT dept_id FROM Department WHERE dept_name = 'IT');
```

-- 4) Delete employee records whose salary is greater than 100000 [cite: 50]

```
DELETE FROM Employee WHERE salary > 100000;
```

-- 5) Display employee name and salary after deducting 5% tax [cite: 51]

```
SELECT emp_name, salary - (salary * 0.05) AS salary_after_tax FROM Employee;
```

-- 6) Display employees who do not live in 'Mumbai' and whose names start with a [cite: 52]

```
SELECT * FROM Employee WHERE city != 'Mumbai' AND emp_name LIKE 'A%';
```

-- 7) Display employees who belong to 'IT' department and have salary greater than 50000 [cite: 53]

```
SELECT * FROM Employee
```

```
WHERE salary > 50000 AND department = (SELECT dept_id FROM Department WHERE dept_name = 'IT');
```

Set 5

Queries

-- 1) Create a new user named with Class_div_rollno (Assuming user 'TE_A_01') [cite: 55]

```
CREATE USER TE_A_01 IDENTIFIED BY password123;
```

-- 2) Grant SELECT permission on the Employee table to a user named Class_div_rollno [cite: 56]

```
GRANT SELECT ON Employee TO TE_A_01;
```

-- 3) Grant INSERT and UPDATE permissions on the Employee table to Class_div_rollno [cite: 56]

```
GRANT INSERT, UPDATE ON Employee TO TE_A_01;
```

-- 4) Revoke UPDATE permission from user1 (assuming TE_A_01 is user1) [cite: 57]

```
REVOKE UPDATE ON Employee FROM TE_A_01;
```

-- 5) Create a role named manager_role [cite: 58]

```
CREATE ROLE manager_role;
```

-- 6) Grant SELECT and DELETE privileges on Employee table to manager_role [cite: 59]

```
GRANT SELECT, DELETE ON Employee TO manager_role;
```

-- 7) Revoke the role manager_role from user2 (assuming user2 exists) [cite: 60]

```
REVOKE manager_role FROM user2;
```

Set 6

Queries

-- 1) Create EmpTest from Emp table by copying structure and data [cite: 107]

```
CREATE TABLE EmpTest AS SELECT * FROM Employees;
```

-- 2) Add a new row into the EmpTest table for Empno, Ename, Sal columns. In Ename should have the Current User. [cite: 108, 109]

```
INSERT INTO EmpTest (EMPLOYEE_ID, FIRST_NAME, SALARY, LAST_NAME, EMAIL, HIRE_DATE, JOB_ID)
```

```
VALUES (999, USER, 50000, 'UserLastName', 'user@company.com', SYSDATE, 'IT_PROG');
```

-- 3) Update EmpTest by increasing the Salary of Vaishali by 15%. Confirm your changes. [cite: 110]

```
UPDATE EmpTest SET SALARY = SALARY * 1.15 WHERE FIRST_NAME = 'Vaishali';
```

```
COMMIT;
```

-- 4) Increase all the employees salary by 10% in EmpTest table who are working in PUNE [cite: 111]

```
UPDATE EmpTest
```

```
SET SALARY = SALARY * 1.10
```

```
WHERE DEPARTMENT_ID IN (
```

```
    SELECT DEPARTMENT_ID FROM Departments d
```

```
    JOIN Locations l ON d.LOCATION_ID = l.LOCATION_ID
```

```
    WHERE l.CITY = 'PUNE'
```

```
);
```

-- 5) Delete all the Comm data from EmpTest Table [cite: 112]

```
UPDATE EmpTest SET COMMISSION_PCT = NULL;
```

-- 6) Delete all the employees in deptno 10 and do not issue a Commit [cite: 113]

```
DELETE FROM EmpTest WHERE DEPARTMENT_ID = 10;
```

-- 7) Issue a Rollback and reflect changes [cite: 114]

ROLLBACK;

SELECT * FROM EmpTest;

Set 7

Queries

-- 1) Start a transaction and insert a new employee record. Commit the transaction. [cite: 119]

INSERT INTO Employees (EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, HIRE_DATE, JOB_ID)

VALUES (1001, 'Sam', 'Smith', 'sam@corp.com', SYSDATE, 'IT_PROG');

COMMIT;

-- 2) Start a transaction, update employee salary, and rollback the transaction. [cite: 120]

UPDATE Employees SET SALARY = 80000 WHERE EMPLOYEE_ID = 1001;

ROLLBACK;

-- 3) Start a transaction, create a savepoint named SP1, update salary, and rollback to SP1. [cite: 121]

SAVEPOINT SP1;

UPDATE Employees SET SALARY = 90000 WHERE EMPLOYEE_ID = 1001;

ROLLBACK TO SP1;

-- 4) Demonstrate the use of multiple savepoints within a single transaction. [cite: 122]

UPDATE Employees SET SALARY = 60000 WHERE EMPLOYEE_ID = 1001;

SAVEPOINT SP2;

UPDATE Employees SET SALARY = 70000 WHERE EMPLOYEE_ID = 1001;

SAVEPOINT SP3;

ROLLBACK TO SP2;

COMMIT;

-- 5) Start a transaction, delete an employee record, and commit the changes. [cite: 123]

DELETE FROM Employees WHERE EMPLOYEE_ID = 1001;

COMMIT;

Set 8

Queries

-- 1) Find the highest, lowest, sum and average salary of all employees. Label the columns. [cite: 127]

SELECT

MAX(SALARY) AS "Highest",

```
MIN(SALARY) AS "Lowest",  
SUM(SALARY) AS "Sum",  
AVG(SALARY) AS "Average"  
FROM Employees;
```

-- 2) write a query to display the number of people with the same job. [cite: 128]

```
SELECT JOB_ID, COUNT(*) AS "Number of People"  
FROM Employees  
GROUP BY JOB_ID;
```

-- 3) Determine the number of managers without listing them. Label the column number of Managers [cite: 129]

```
SELECT COUNT(DISTINCT MANAGER_ID) AS "Number of Managers"  
FROM Employees;
```

-- 4) Find the difference between the highest and lowest salaries. Label the column DIFFERENCE. [cite: 130]

```
SELECT MAX(SALARY) - MIN(SALARY) AS "DIFFERENCE"  
FROM Employees;
```

-- 5) Create a report to display the manager number and the salary of the lowest-paid employee for that manager. [cite: 131]

```
SELECT MANAGER_ID, MIN(SALARY)  
FROM Employees  
WHERE MANAGER_ID IS NOT NULL  
GROUP BY MANAGER_ID;
```

Set 9

Assume a basic Employee (emp_id, emp_name, department, city, salary) table structure.

Queries

-- 1) Display department-wise total salary of employees. [cite: 135]

```
SELECT department, SUM(salary) FROM Employee GROUP BY department;
```

-- 2) Display department-wise average salary. [cite: 136]

```
SELECT department, AVG(salary) FROM Employee GROUP BY department;
```

-- 3) City-wise employee count greater than 3. [cite: 137]

```
SELECT city, COUNT(*) FROM Employee GROUP BY city HAVING COUNT(*) > 3;
```

-- 4) Department-wise maximum salary above 60,000. [cite: 138]

```
SELECT department, MAX(salary) FROM Employee GROUP BY department HAVING
```



```
MAX(salary) > 60000;
```

```
-- 5) Display department and city-wise total salary. [cite: 139]
```

```
SELECT department, city, SUM(salary) FROM Employee GROUP BY department, city;
```

```
-- 6) Display departments having more than one employee. [cite: 140]
```

```
SELECT department, COUNT(*) FROM Employee GROUP BY department HAVING COUNT(*) > 1;
```

```
-- 7) Display city-wise employee count in descending order. [cite: 141]
```

```
SELECT city, COUNT(*) AS emp_count FROM Employee GROUP BY city ORDER BY emp_count  
DESC;
```

Set 10

Utilizing the same basic Employee structure.

Queries

```
-- 1) Department-wise total salary greater than 2,00,000. [cite: 145]
```

```
SELECT department, SUM(salary) FROM Employee GROUP BY department HAVING  
SUM(salary) > 200000;
```

```
-- 2) Department-wise average salary more than 40,000. [cite: 146]
```

```
SELECT department, AVG(salary) FROM Employee GROUP BY department HAVING  
AVG(salary) > 40000;
```

```
-- 3) Display city-wise number of employees. [cite: 147]
```

```
SELECT city, COUNT(*) FROM Employee GROUP BY city;
```

```
-- 4) Display department-wise maximum salary. [cite: 148]
```

```
SELECT department, MAX(salary) FROM Employee GROUP BY department;
```

```
-- 5) Display city-wise average salary. [cite: 149]
```

```
SELECT city, AVG(salary) FROM Employee GROUP BY city;
```

```
-- 6) Display department-wise employee count where the number of employees is more than 3. [cite: 150]
```

```
SELECT department, COUNT(*) FROM Employee GROUP BY department HAVING COUNT(*) > 3;
```

```
-- 7) Display city-wise employee count in ascending order. [cite: 151]
```

```
SELECT city, COUNT(*) AS emp_count FROM Employee GROUP BY city ORDER BY emp_count  
ASC;
```

Here are the SQL and PL/SQL solutions for problem statements 11 through 20 based on the document you provided.

Set 11

Queries

-- 1) Create a table dept1 as below with following columns

```
CREATE TABLE dept1 (  
    DEPTNO INTEGER PRIMARY KEY,  
    dname VARCHAR2(30) NOT NULL,  
    loc VARCHAR2(30) NOT NULL  
);
```

-- 2) Add column mgr as number in the dept table (assuming dept1)

```
ALTER TABLE dept1 ADD mgr NUMBER;
```

-- 3) Create table empl with following columns

```
CREATE TABLE empl (  
    empno INTEGER,  
    ename VARCHAR2(20),  
    sal NUMBER(10,2),  
    deptno NUMBER  
);
```

-- 4) Add constraints on empl table

```
ALTER TABLE empl ADD CONSTRAINT pk_empno PRIMARY KEY (empno);  
ALTER TABLE empl MODIFY ename VARCHAR2(20) NOT NULL;  
ALTER TABLE empl ADD CONSTRAINT chk_sal CHECK (sal > 10000);  
ALTER TABLE empl ADD CONSTRAINT fk_deptno FOREIGN KEY (deptno) REFERENCES  
dept1(DEPTNO);
```

-- 5) Create table dept11, emp11 from dept1, empl

```
CREATE TABLE dept11 AS SELECT * FROM dept1;  
CREATE TABLE emp11 AS SELECT * FROM empl;
```

-- 6) Add a new column address varchar2(30) in the empl table

```
ALTER TABLE empl ADD address VARCHAR2(30);
```

-- 7) Rename sal column in empl table to salary

```
ALTER TABLE empl RENAME COLUMN sal TO salary;
```

-- 8) Display constraints names and verify the names for the empl

```
SELECT constraint_name, constraint_type, search_condition  
FROM user_constraints  
WHERE table_name = 'EMPL';
```

Set 12

Note: The problem description references classic Oracle EMP/DEPT columns (like ENAME, LOC) but provides the standard HR Employees and Departments schema. The queries below map to the provided HR schema while providing the requested aliases.

Queries

-- 1) Write a query for the HR department to produce the addresses of all the departments. Use Natural Join.

```
SELECT EMPLOYEE_ID AS EMPNO, FIRST_NAME AS ENAME, SALARY AS SAL,  
       DEPARTMENT_NAME AS DNAME, LOCATION_ID AS LOC  
FROM Employees  
NATURAL JOIN Departments;
```

-- 2) Write a query to display the JOB, MGR, SAL, COMM, DNAME of employees whose JOB is SALESMAN.

```
SELECT e.JOB_ID AS JOB, e.MANAGER_ID AS MGR, e.SALARY AS SAL,  
       e.COMMISSION_PCT AS COMM, d.DEPARTMENT_NAME AS DNAME  
FROM Employees e  
JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID  
WHERE e.JOB_ID = 'SALESMAN' OR e.JOB_ID LIKE '%SA_REP%';
```

-- 3) Display the ENAME, job, DEPTNO and DNAME for all employees who work in DALLAS.
-- (Assuming a Locations table is joined to resolve the city name)

```
SELECT e.FIRST_NAME AS ENAME, e.JOB_ID AS job, e.DEPARTMENT_ID AS DEPTNO,  
       d.DEPARTMENT_NAME AS DNAME  
FROM Employees e  
JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID  
JOIN Locations l ON d.LOCATION_ID = l.LOCATION_ID  
WHERE l.CITY = 'DALLAS';
```

-- 4) Modify the previous Query to display all employees including King, who has no manager. Order by employee number.

```
SELECT e.FIRST_NAME AS ENAME, e.JOB_ID AS job, e.DEPARTMENT_ID AS DEPTNO,  
       d.DEPARTMENT_NAME AS DNAME  
FROM Employees e  
LEFT JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID  
LEFT JOIN Locations l ON d.LOCATION_ID = l.LOCATION_ID  
WHERE l.CITY = 'DALLAS' OR e.MANAGER_ID IS NULL  
ORDER BY e.EMPLOYEE_ID;
```

Set 13

Queries

-- 1) Create a query that displays the name, job, department name, salary and grade for all employees.

-- (Assuming a SALGRADE table exists with GRADE, LOSAL, and HISAL columns)

```
SELECT e.FIRST_NAME, e.JOB_ID, d.DEPARTMENT_NAME, e.SALARY, s.GRADE
FROM Employees e
JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
JOIN SALGRADE s ON e.SALARY BETWEEN s.LOSAL AND s.HISAL;
```

-- 2) Display the ENAME, SAL, MGR, DNAME of employees whose salary is more than 2000. Use the ON clause.

```
SELECT e.FIRST_NAME AS ENAME, e.SALARY AS SAL, e.MANAGER_ID AS MGR,
d.DEPARTMENT_NAME AS DNAME
FROM Employees e
JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
WHERE e.SALARY > 2000;
```

-- 3) Display the EMPNO, ENAME, JOB, DEPTNO, DNAME, LOC of employees. Use left outer join.

```
SELECT e.EMPLOYEE_ID AS EMPNO, e.FIRST_NAME AS ENAME, e.JOB_ID AS JOB,
e.DEPARTMENT_ID AS DEPTNO, d.DEPARTMENT_NAME AS DNAME, d.LOCATION_ID
AS LOC
FROM Employees e
LEFT OUTER JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID;
```

-- 4) Display the Ename, Dname of employees. Use right outer join.

```
SELECT e.FIRST_NAME AS Ename, d.DEPARTMENT_NAME AS Dname
FROM Employees e
RIGHT OUTER JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID;
```

-- 5) Display the EMPNO, DNAME, LOC of employees. Use full outer join.

```
SELECT e.EMPLOYEE_ID AS EMPNO, d.DEPARTMENT_NAME AS DNAME, d.LOCATION_ID
AS LOC
FROM Employees e
FULL OUTER JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID;
```

Set 14

Queries

-- 1) Write a query to display the JOB, MGR, SAL, COMM, DNAME of employees whose JOB is SALESMAN.

```
SELECT e.JOB_ID AS JOB, e.MANAGER_ID AS MGR, e.SALARY AS SAL,
e.COMMISSION_PCT AS COMM, d.DEPARTMENT_NAME AS DNAME
FROM Employees e
JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
WHERE e.JOB_ID = 'SALESMAN';
```

-- 2) Display the ENAME, job, DEPTNO and DNAME for all employees who work in Pune.

```
SELECT e.FIRST_NAME AS ENAME, e.JOB_ID AS job, e.DEPARTMENT_ID AS DEPTNO,
d.DEPARTMENT_NAME AS DNAME
FROM Employees e
JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
```

```
JOIN Locations l ON d.LOCATION_ID = l.LOCATION_ID
```

```
WHERE l.CITY = 'Pune';
```

-- 3 & 4) Display the Ename, Dname of all the employees, Also display those department names which do not have any employees working

```
SELECT e.FIRST_NAME AS Ename, d.DEPARTMENT_NAME AS Dname
```

```
FROM Departments d
```

```
LEFT JOIN Employees e ON d.DEPARTMENT_ID = e.DEPARTMENT_ID;
```

-- 5) Display the EMPNO, ENAME, LOC of those employees who are working as CLERK

```
SELECT e.EMPLOYEE_ID AS EMPNO, e.FIRST_NAME AS ENAME, d.LOCATION_ID AS LOC
```

```
FROM Employees e
```

```
JOIN Departments d ON e.DEPARTMENT_ID = d.DEPARTMENT_ID
```

```
WHERE e.JOB_ID LIKE '%CLERK%';
```

Set 15

PL/SQL Block

```
DECLARE
```

```
    v_empno EMP.EMPNO%TYPE := &Enter_EMPNO;
```

```
    v_ename EMP.ENAME%TYPE := '&Enter_ENAME';
```

```
    v_sal EMP.SAL%TYPE := &Enter_SALARY;
```

```
    v_count NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO v_count FROM EMP WHERE EMPNO = v_empno;
```

```
    IF v_count = 0 THEN
```

```
        INSERT INTO EMP (EMPNO, ENAME, SAL) VALUES (v_empno, v_ename, v_sal);
```

```
        DBMS_OUTPUT.PUT_LINE('Record Inserted.');
```

```
    ELSE
```

```
        UPDATE EMP SET ENAME = v_ename, SAL = v_sal WHERE EMPNO = v_empno;
```

```
        DBMS_OUTPUT.PUT_LINE('Record Updated.');
```

```
    END IF;
```

```
    COMMIT;
```

```
END;
```

```
/
```

Set 16

PL/SQL Block

```
DECLARE
```

```
    v_num NUMBER := &Enter_Number;
```

```
    negative_number_exception EXCEPTION;
```

```
BEGIN
```

```
    IF v_num < 0 THEN
```

```

        RAISE negative_number_exception;
    ELSIF MOD(v_num, 2) = 0 THEN
        DBMS_OUTPUT.PUT_LINE('The number ' || v_num || ' is EVEN.');
```

```

    ELSE
        DBMS_OUTPUT.PUT_LINE('The number ' || v_num || ' is ODD.');
```

```

    END IF;
EXCEPTION
    WHEN negative_number_exception THEN
        DBMS_OUTPUT.PUT_LINE('Error: The number cannot be negative.');
```

```

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');
```

```

END;
/

```

Set 17

PL/SQL Block

```

DECLARE
    v_empno EMP.EMPNO%TYPE := &Enter_EMPNO;
    v_deptno EMP.DEPTNO%TYPE;
BEGIN
    SELECT DEPTNO INTO v_deptno FROM EMP WHERE EMPNO = v_empno;

    IF v_deptno = 10 THEN
        UPDATE EMP SET SAL = SAL * 1.10 WHERE EMPNO = v_empno;
        DBMS_OUTPUT.PUT_LINE('Salary updated by 10% for Dept 10');
```

```

    ELSIF v_deptno = 20 THEN
        UPDATE EMP SET SAL = SAL * 1.15 WHERE EMPNO = v_empno;
        DBMS_OUTPUT.PUT_LINE('Salary updated by 15% for Dept 20');
```

```

    ELSE
        UPDATE EMP SET SAL = SAL + NVL(COMM, 0) WHERE EMPNO = v_empno;
        DBMS_OUTPUT.PUT_LINE('Salary updated with COMM for other Dept');
```

```

    END IF;
    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found.');
```

```

END;
/

```

Set 18

PL/SQL Block

```

DECLARE
    v_roll Stud.Roll%TYPE := &Enter_Roll_No;
    v_att Stud.Att%TYPE;
    invalid_roll EXCEPTION;
BEGIN

```

```

SELECT Att INTO v_att FROM Stud WHERE Roll = v_roll;

IF v_att < 75 THEN
    DBMS_OUTPUT.PUT_LINE('Term not granted');
    UPDATE Stud SET Status = 'D' WHERE Roll = v_roll;
ELSE
    DBMS_OUTPUT.PUT_LINE('Term granted');
    UPDATE Stud SET Status = 'ND' WHERE Roll = v_roll;
END IF;
COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Roll number does not exist. ');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred. ');
END;
/

```

Set 19

PL/SQL Block

```

DECLARE
    v_roll Borrower.Rollin%TYPE := &Enter_Roll_No;
    v_book Borrower.NameofBook%TYPE := '&Enter_Book_Name';
    v_dateofissue Borrower.Dateofissue%TYPE;
    v_days NUMBER;
    v_fineAmt NUMBER := 0;
BEGIN
    SELECT Dateofissue INTO v_dateofissue
    FROM Borrower
    WHERE Rollin = v_roll AND NameofBook = v_book AND Status = 'I';

    v_days := TRUNC(SYSDATE) - v_dateofissue;

    IF v_days BETWEEN 15 AND 30 THEN
        v_fineAmt := (v_days - 14) * 5;
    ELSIF v_days > 30 THEN
        v_fineAmt := (16 * 5) + ((v_days - 30) * 50);
    END IF;

    UPDATE Borrower SET Status = 'R' WHERE Rollin = v_roll AND NameofBook = v_book;

    IF v_fineAmt > 0 THEN
        INSERT INTO Fine (Roll_no, Date, Amt) VALUES (v_roll, SYSDATE, v_fineAmt);
        DBMS_OUTPUT.PUT_LINE('Fine of Rs. ' || v_fineAmt || ' applied and stored. ');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Book returned with no fine. ');
    END IF;
    COMMIT;
EXCEPTION

```

```

WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No active issue found for this Roll Number and Book.');
```

END;

/

Set 20

Table Creation Assumptions

```

-- Assuming schemas based on typical University DBs:
-- student(S_ID, name, dept_name, tot_cred)
-- instructor(T_ID, name, dept_name, salary)
-- course(course_id, title, dept_name, credits)
-- takes(S_ID, course_id) <-- Implicitly required for query 'v'
```

Queries

```

-- i) Find the average salary of instructor in those departments where the average salary is more than Rs.
42000/-
```

```

SELECT dept_name, AVG(salary) AS Avg_Salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

```

-- ii) Increase the salary of each instructor in the computer department by 10%.
```

```

UPDATE instructor
SET salary = salary * 1.10
WHERE dept_name = 'Computer';
```

```

-- iii) Find the names of instructors whose names are neither 'Amol' nor 'Amit'.
```

```

SELECT name
FROM instructor
WHERE name NOT IN ('Amol', 'Amit');
```

```

-- iv) Find the names of student which contains 'am' as its substring.
```

```

SELECT name
FROM student
WHERE LOWER(name) LIKE '%am%';
```

```

-- v) Find the name of students from computer department that "DBMS" courses they take.
```

```

SELECT s.name
FROM student s
JOIN takes t ON s.S_ID = t.S_ID
JOIN course c ON t.course_id = c.course_id
WHERE s.dept_name = 'Computer' AND c.title = 'DBMS';
```


Here are the SQL, PL/SQL, and MongoDB query solutions for problem statements 21 through 35 based on the provided document.

Set 21

PL/SQL Block: Top N Salaries

```
DECLARE
    v_n NUMBER := &Enter_Top_N_Value;
    CURSOR c_emp IS
        SELECT first_name, salary
        FROM employees
        ORDER BY salary DESC;
    v_name employees.first_name%TYPE;
    v_sal employees.salary%TYPE;
    v_count NUMBER := 1;
BEGIN
    OPEN c_emp;
    DBMS_OUTPUT.PUT_LINE('Top ' || v_n || ' Salaries:');
    LOOP
        FETCH c_emp INTO v_name, v_sal;
        EXIT WHEN c_emp%NOTFOUND OR v_count > v_n;
        DBMS_OUTPUT.PUT_LINE(v_count || '. ' || v_name || ' - ' || v_sal);
        v_count := v_count + 1;
    END LOOP;
    CLOSE c_emp;
END;
```

Set 22

Stored Procedure & Execution Block

```
-- 1. Create the Procedure
CREATE OR REPLACE PROCEDURE proc_Grade (
    p_name IN VARCHAR2,
    p_marks IN NUMBER,
    p_roll IN NUMBER
) IS
    v_class VARCHAR2(30);
BEGIN
    IF p_marks <= 1500 AND p_marks >= 990 THEN
        v_class := 'Distinction';
    ELSIF p_marks BETWEEN 900 AND 989 THEN
        v_class := 'First Class';
    ELSIF p_marks BETWEEN 825 AND 899 THEN
        v_class := 'Higher Second Class';
    ELSE
        v_class := 'Other / Pass Class';
    END IF;
    -- Insert into Result table
    INSERT INTO Result (Roll, Name, Class)
```

```

VALUES (p_roll, p_name, v_class);
END proc_Grade;
/

-- 2. PL/SQL Block to call the procedure
DECLARE
    v_roll NUMBER := 1;
BEGIN
    FOR rec IN (SELECT name, total_marks FROM Stud_Marks) LOOP
        proc_Grade(rec.name, rec.total_marks, v_roll);
        v_roll := v_roll + 1;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Grades calculated and stored successfully.');
```

Set 23

SQL Queries

```

-- 1. List employees whose salary is greater than the salary of ALLEN
SELECT * FROM employees
WHERE salary > (SELECT salary FROM employees WHERE first_name = 'ALLEN' OR last_name = 'ALLEN');
```

```

-- 2. Find employees whose job is the same as that of MARTIN
SELECT * FROM employees
WHERE job_id = (SELECT job_id FROM employees WHERE first_name = 'MARTIN' OR last_name = 'MARTIN');
```

```

-- 3. Display employees who are not working in the department of KING
SELECT * FROM employees
WHERE department_id != (SELECT department_id FROM employees WHERE first_name = 'KING' OR last_name = 'KING');
```

```

-- 4. Find employees who earn more than the salary of BLAKE
SELECT * FROM employees
WHERE salary > (SELECT salary FROM employees WHERE first_name = 'BLAKE' OR last_name = 'BLAKE');
```

```

-- 5. Display employees hired after JONES
SELECT * FROM employees
WHERE hire_date > (SELECT hire_date FROM employees WHERE first_name = 'JONES' OR last_name = 'JONES');
```

```

-- 6. Find employees whose department number is the same as CLARK
SELECT * FROM employees
WHERE department_id = (SELECT department_id FROM employees WHERE first_name = 'CLARK' OR last_name = 'CLARK');
```

Set 24

SQL Queries

```
-- 1. Find employees whose salary is greater than ANY employee in department 30
SELECT * FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 30);

-- 2. Display employees whose salary is greater than ALL employees in department 20
SELECT * FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 20);

-- 3. List employees working in departments where the job is MANAGER
SELECT * FROM employees
WHERE department_id IN (SELECT department_id FROM employees WHERE job_id LIKE
'%MGR%' OR job_id LIKE '%MANAGER%');

-- 4. Find employees whose salary is greater than the average salary of their own department
SELECT e.first_name, e.salary, e.department_id
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id =
e.department_id);

-- 5. Display employees who earn the highest salary in their department
SELECT first_name, salary, department_id
FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees WHERE department_id =
e.department_id);

-- 6. Find employees who have the same job as at least one employee in department 20
SELECT * FROM employees
WHERE job_id IN (SELECT job_id FROM employees WHERE department_id = 20);
```

Set 25

PL/SQL Trigger on Library Table

```
-- Assuming a Library table (Book_ID, Title, Author) and Library_Audit (Book_ID, Old_Title, Action_Date,
Action_Type)
CREATE OR REPLACE TRIGGER trg_library_audit
BEFORE UPDATE OR DELETE ON Library
FOR EACH ROW
BEGIN
    IF UPDATING THEN
        INSERT INTO Library_Audit (Book_ID, Old_Title, Action_Date, Action_Type)
        VALUES (:OLD.Book_ID, :OLD.Title, SYSDATE, 'UPDATE');
    ELSIF DELETING THEN
        INSERT INTO Library_Audit (Book_ID, Old_Title, Action_Date, Action_Type)
        VALUES (:OLD.Book_ID, :OLD.Title, SYSDATE, 'DELETE');
    END IF;
END;
/
```

Set 26 (MongoDB)

MongoDB Queries

```
// 1. Find the average price for each customers having status 'A'
db.orderinfo.aggregate([
  { $match: { status: "A" } },
  { $group: { _id: "$cust_id", avgPrice: { $avg: "$price" } } }
]);

// 2. Display the status of the customers whose amount/price lie between 100 and 1000
db.orderinfo.find({ price: { $gte: 100, $lte: 1000 } }, { status: 1, _id: 0 });

// 3. Display the customers information without "_id"
db.orderinfo.find({}, { _id: 0 });

// 4. Create a simple index on orderinfo collection
db.orderinfo.createIndex({ cust_id: 1 });
```

Set 27 (MongoDB)

MongoDB Queries

```
// i. Find the name of the movie having budget greater than 1,00,000
db.movies.find({ budget: { $gt: 100000 } }, { name: 1, _id: 0 });

// ii. Find the name of producer who lives in Pune
db.movies.find({ "producer.address": "PUNE" }, { "producer.name": 1, _id: 0 });

// iii. Update the type of movie "action" to "horror"
db.movies.updateMany({ type: "action" }, { $set: { type: "horror" } });

// iv. Find all the documents produced by name "producer1" with their address
db.movies.find({ "producer.name": "producer1" }, { "producer.name": 1, "producer.address": 1, _id: 0 });
```

Set 28 (MongoDB)

MongoDB Queries

```
// 1. Insert sample documents
db.Teachers.insertMany([{ Tname: "Mr. A", dno: 2, Experience: 5, Salary: 15000, Date_of_Joining:
new Date() }]);
db.Students.insertMany([{ Sname: "xyz", Roll_No: 2, Class: "FE" }, { Sname: "John", Roll_No: 5,
Class: "SE" }]);

// 2. Find info about all teachers of Dno = 2 and having salary >= 10,000
```

```

db.Teachers.find({ dno: 2, Salary: { $gte: 10000 } });

// 3. Find the student information having Roll_no = 2 or Sname = 'xyz'
db.Students.find({ $or: [{ Roll_No: 2 }, { Sname: "xyz" }] });

// 4. Update student name whose Roll_No = 5
db.Students.updateOne({ Roll_No: 5 }, { $set: { Sname: "Updated Name" } });

// 5. Delete all student whose Class is 'FE'
db.Students.deleteMany({ Class: "FE" });

// 6. Apply index on Students Collection
db.Students.createIndex({ Roll_No: 1 });

```

Set 29 (MongoDB)

MongoDB Queries

```

// 1 & 2. Database `department`, collection `teacher` already in use context.
use department;

// 3. Display the department wise average salary.
db.teacher.aggregate([ { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } } ]);

// 4. Display the no. Of employees working in each department.
db.teacher.aggregate([ { $group: { _id: "$department", empCount: { $sum: 1 } } } ]);

// 5. Display the department wise minimum salary.
db.teacher.aggregate([ { $group: { _id: "$department", minSalary: { $min: "$salary" } } } ]);

// 6. Apply index and drop index
db.teacher.createIndex({ department: 1 });
db.teacher.dropIndex("department_1");

```

Set 30 (MongoDB)

MongoDB Queries

```

use product_detail; // 1
show dbs; // 2
db.createCollection("product"); // 3
show collections; // 4

// 5. Insert documents
db.product.insertMany([
  { "item": "pen", "price": 10, "quantity": 2 },
  { "item": "book", "price": 50, "quantity": 1 },
  { "item": "pen", "price": 10, "quantity": 5 },
  { "item": "pencil", "price": 5, "quantity": 10 }
]);

```

```

});
db.product.find(); // 6. Retrieve documents
db.product.createIndex({ item: 1 }); // 7. Create index on item asc
db.product.getIndexes(); // 8. Show index
db.product.createIndex({ price: 1, quantity: -1 }); // 9. Index price asc, qty desc
db.product.dropIndex({ price: 1, quantity: -1 }); // 10. Drop index on price/qty

```

Set 31 (MongoDB)

MongoDB Queries

```

// i. Display the name of the customer having the price between 250 and 450
db.orderinfo.find({ price: { $gte: 250, $lte: 450 } }, { cust_name: 1, _id: 0 });

// ii. Increment price by 10 for cust_id: 123 and decrement price by 5 for cust_id: 124
db.orderinfo.updateOne({ cust_id: 123 }, { $inc: { price: 10 } });
db.orderinfo.updateOne({ cust_id: 124 }, { $inc: { price: -5 } });

// iii. Remove any one of the field from the orderinfo collection (e.g., status)
db.orderinfo.updateMany({}, { $unset: { status: "" } });

// iv. Find the name of the customer whose status is either A or price is 250 or both.
db.orderinfo.find({ $or: [{ status: "A" }, { price: 250 } ] }, { cust_name: 1, _id: 0 });

```

Set 32

SQL Queries

```

-- i. Modify the database so that a particular company (eg. ABC) now in Pune
UPDATE Company SET city = 'Pune' WHERE cname = 'ABC';

-- ii. Give all managers of Mbank a 10% raise. If salary is > 20000, give only 3% raise.
UPDATE works
SET sal = CASE
    WHEN sal > 20000 THEN sal * 1.03
    ELSE sal * 1.10
END
WHERE cname = 'Mbank'
AND emp_id IN (SELECT mgr_id FROM Manager);

-- iii. Find out the names of all the employees who works in 'Bosch' company in city Pune
SELECT w.ename

```

```
FROM works w
JOIN Company c ON w.c_id = c.c_id
WHERE w.cname = 'Bosch' AND c.city = 'Pune';
```

-- iv. Delete all records in the works table for employees of SBC Company whose salary > 50000

```
DELETE FROM works
WHERE cname = 'SBC Company' AND sal > 50000;
```

Set 33 (MongoDB)

MongoDB Queries

```
use Employee; // 1
db.createCollection("emp1"); // 2
db.emp1.insertMany([ // 3
  {eno: 1, ename: "neha", address: "Pune", sal: 12000},
  {eno: 2, ename: "amit", address: "Mumbai", sal: 6000},
  {eno: 3, ename: "nilesh", address: "Delhi", sal: 4000}
]);
db.emp1.find(); // 4
db.emp1.find({ sal: { $gt: 5000 } }); // 5
db.emp1.find({ sal: { $lt: 15000 } }); // 6
db.emp1.find({ sal: { $gt: 10000, $lt: 20000 } }); // 7
db.emp1.updateMany({}, { $mul: { sal: 1.10 } }); // 8
db.emp1.deleteMany({ sal: { $lt: 5000 } }); // 9
db.emp1.renameCollection("employee1"); // 10
db.employee1.find({ ename: /^n/i }); // 11
db.employee1.find().sort({ ename: 1 }); // 12
use Employee1; // 13
db.dropDatabase(); // 14
use Employee;
db.emp1.insert({ eno: 4, ename: "raj" }); // 15
db.emp1.drop(); // 16
```

Set 34

SQL Queries

-- 1. Find the names of employees who work for First Bank Corporation.

```
SELECT employee_name FROM Works WHERE company_name = 'First Bank Corporation';
```

-- 2. Find the names and cities of residence of all employees who work for First Bank Corporation.

```
SELECT e.employee_name, e.city
FROM Employee e
JOIN Works w ON e.employee_name = w.employee_name
```

```
WHERE w.company_name = 'First Bank Corporation';
```

```
-- 3. Names, street addresses, and cities... First Bank... earn more than $10000.
```

```
SELECT e.employee_name, e.street, e.city
```

```
FROM Employee e
```

```
JOIN Works w ON e.employee_name = w.employee_name
```

```
WHERE w.company_name = 'First Bank Corporation' AND w.salary > 10000;
```

```
-- 4. Find all employees who live in the same cities as the companies for which they work.
```

```
SELECT e.employee_name
```

```
FROM Employee e
```

```
JOIN Works w ON e.employee_name = w.employee_name
```

```
JOIN Company c ON w.company_name = c.company_name
```

```
WHERE e.city = c.city;
```

```
-- 5. Employees who live in the same cities and on the same streets as do their manager.
```

```
SELECT e1.employee_name
```

```
FROM Employee e1
```

```
JOIN Manages m ON e1.employee_name = m.employee_name
```

```
JOIN Employee e2 ON m.manager_name = e2.employee_name
```

```
WHERE e1.city = e2.city AND e1.street = e2.street;
```

```
-- 6. Find all employees who do not work for First Bank Corporation.
```

```
SELECT employee_name FROM Works WHERE company_name != 'First Bank Corporation';
```

```
-- 7. Give all employees of First Bank Corporation a 10% raise.
```

```
UPDATE Works SET salary = salary * 1.10 WHERE company_name = 'First Bank Corporation';
```

```
-- 8. Delete all tuples in the "Works" relation for employees of "Small Bank Corporation".
```

```
DELETE FROM Works WHERE company_name = 'Small Bank Corporation';
```

Set 35

SQL Queries

```
-- 1) Find all employees in the database who earn more than each employee of Small Bank Corporation.
```

```
SELECT employee_name FROM Works
```

```
WHERE salary > ALL (SELECT salary FROM Works WHERE company_name = 'Small Bank Corporation');
```

```
-- 2) Find all companies located in every city in which Small Bank Corporation is located.
```

```
SELECT company_name FROM Company c1
```

```
WHERE NOT EXISTS (
```

```
    SELECT city FROM Company WHERE company_name = 'Small Bank Corporation'
```

```
    MINUS
```

```
    SELECT city FROM Company c2 WHERE c1.company_name = c2.company_name
```

```
);
```

```
-- 3) Find all employees who earn more than the average salary of all employees of their companies.
```

```
SELECT w1.employee_name
```

```
FROM Works w1
```

```
WHERE salary > (SELECT AVG(salary) FROM Works w2 WHERE w1.company_name = w2.company_name);
```


-- 4) Find the company that has the most employees.

```
SELECT company_name
FROM Works
GROUP BY company_name
HAVING COUNT(*) = (
    SELECT MAX(COUNT(*)) FROM Works GROUP BY company_name
);
```

-- 5) Find the company that has the smallest payroll.

```
SELECT company_name
FROM Works
GROUP BY company_name
HAVING SUM(salary) = (
    SELECT MIN(SUM(salary)) FROM Works GROUP BY company_name
);
```

-- 6) Find those companies whose employees earn a higher salary, on average, than the average salary at First Bank Corporation.

```
SELECT company_name
FROM Works
GROUP BY company_name
HAVING AVG(salary) > (
    SELECT AVG(salary) FROM Works WHERE company_name = 'First Bank Corporation'
);
```

-- 7) Modify the database so that "Jones" now lives in Newtown.

```
UPDATE Employee SET city = 'Newtown' WHERE employee_name = 'Jones';
```